Allocation Patterns

- **Map-Reduce**
- Multi-Tier

Map-Reduce Pattern

- **Context:** Businesses have a pressing need to quickly analyze enormous volumes of data they generate or access, at petabyte scale (i.e. logs of interactions in a social network site, massive document or data repositories). Programs for the analysis of this data should be easy to write, run efficiently, and be resilient with respect to hardware failure.

- **Problem:** For many applications with ultra-large data sets, sorting the data and then analyzing the grouped data is sufficient. The problem the map-reduce pattern solves is to efficiently perform a distributed and parallel sort of a large data set and provide a simple means for the programmer to specify the analysis to be done.
- **Solution:** The map-reduce pattern requires three parts:
  - A **specialized infrastructure** takes care of allocating software to the hardware nodes in a massively parallel computing environment and handles sorting the data as needed.
  - A programmer specified component called **the map** which filters the data to retrieve those items to be combined.
  - A programmer specified component called **reduce** which combines the results of the map

Map-Reduce Solution - 1

- Overview: The map-reduce pattern provides a framework for analyzing a large distributed set of data that will execute in parallel, on a set of processors. This parallelization allows for low latency and high availability. The map performs the extract and transform portions of the analysis and the reduce performs the loading of the results.

- **Elements**:
  - Map is a function with multiple instances deployed across multiple processors that performs the extract and transformation portions of the analysis.
  - Reduce is a function that may be deployed as a single instance or as multiple instances across processors to perform the load portion of extract-transform-load.
  - The infrastructure is the framework responsible for deploying map and reduce instances, shepherding the data between them, and detecting and recovering from failure.

Map-Reduce Solution - 2

- **Relations**:
  - Deploy on is the relation between an instance of a map or reduce function and the processor onto which it is installed.
  - Instantiate, monitor, and control is the relation between the infrastructure and the instances of map and reduce.

Map-Reduce Generic Example

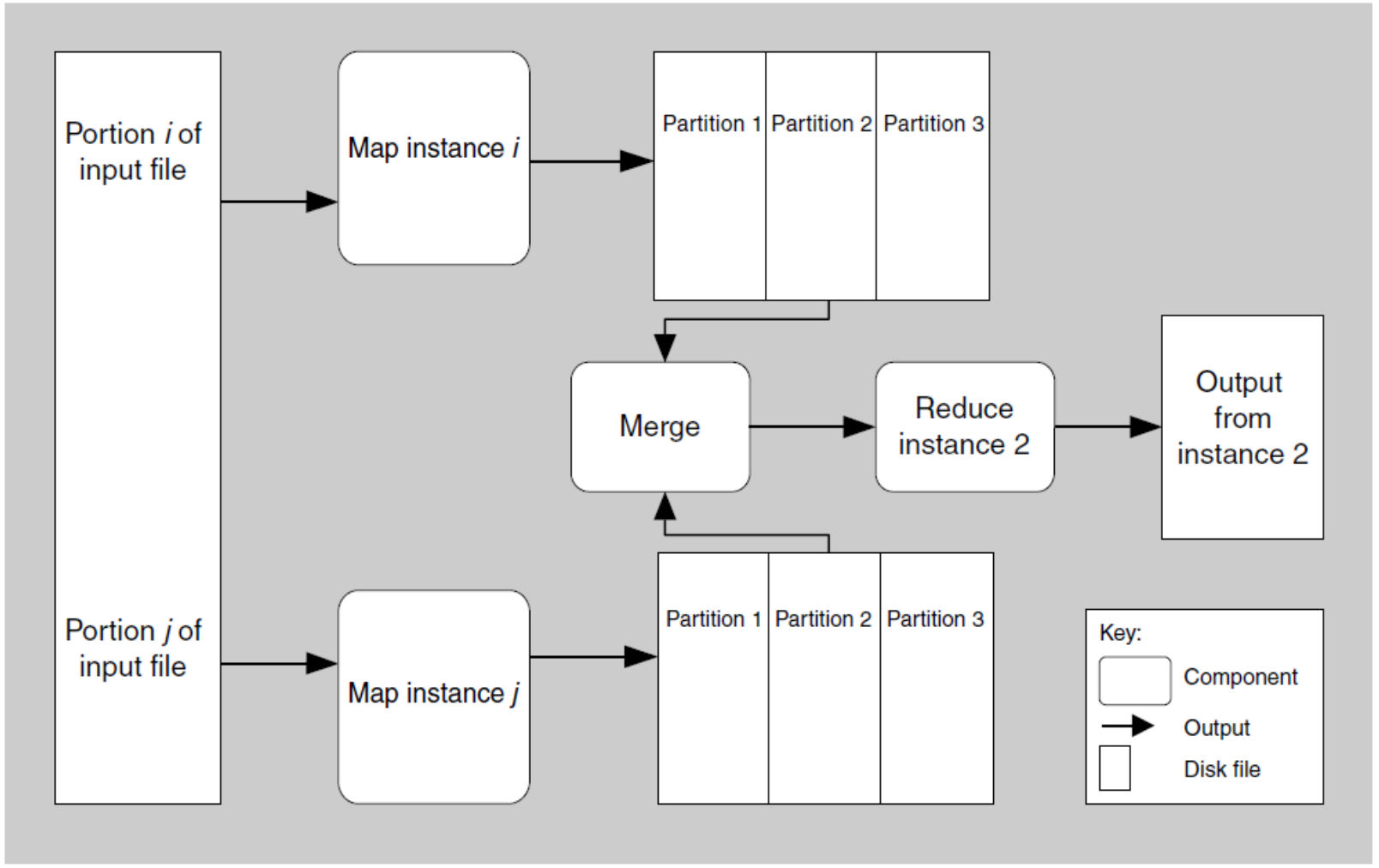

FIGURE 13.14 A component-and-connector view of map-reduce showing how the data processed by map is partitioned and subsequently processed by reduce

Map-Reduce Real Example: Counting word occurrences in a document

- This example can be carried out with a single map function.
- The document is the data set. The map function will find every word in the document and output a <word, 1> pair for each.
- For example, if the document begins with the words “Having a whole book . . . ,” then the first results of map will be

<Having, 1>

<a, 1>

<whole, 1>

<book, 1>

Map-Reduce Real Example: Counting word occurrences in a document

- The reduce function will take that list in sorted order, add up the 1s for each word to get a count, and output the result.

Map-Reduce Solution - 2

- **Constraints**:
  - The data to be analyzed must exist as a set of files.
  - Map functions are stateless and do not communicate with each other.
  - The only communication between map reduce instances is the data emitted from the map instances as <key, value> pairs.

- **Weaknesses**:
  - If you do not have large data sets, the overhead of map-reduce is not justified.
  - If you cannot divide your data set into similar sized subsets, the advantages of parallelism are lost.
  - Operations that require multiple reduces are complex to orchestrate.